Lecture 3: Generative Adversarial Networks (GAN)

적대적 학습을 통한 고품질 이미지 생성

목차

1. 개요
2. Original GAN
    - 2.1 핵심 아이디어: 위조지폐범 vs 경찰
    - 2.2 모델 구조
    - 2.3 목적 함수 (Minimax Game)
    - 2.4 판별자의 관점: Maximize
    - 2.5 생성자의 관점: Minimize
    - 2.6 Non-Saturating Loss (실전 트릭)
    - 2.7 전체 손실 함수 정리
    - 2.8 교대 훈련 (Alternating Training)
    - 2.9 GAN의 이론적 분석: 최적 판별자
    - 2.10 GAN의 문제점
    - 2.11 PyTorch 구현
3. 생성 모델의 성능 평가 지표
    - 3.1 Inception Score (IS)
    - 3.2 Fréchet Inception Distance (FID)
    - 3.3 기타 지표
4. DCGAN (Deep Convolutional GAN)
    - 4.1 핵심 아이디어
    - 4.2 네트워크 구조
    - 4.3 잠재 공간의 산술 연산
    - 4.4 훈련 가이드라인
    - 4.5 PyTorch 구현
5. ProGAN (Progressive Growing of GANs)
    - 5.1 핵심 아이디어: 점진적 해상도 증가
    - 5.2 점진적 레이어 추가 방법
    - 5.3 Mini-Batch Standard Deviation
    - 5.4 기타 기법
    - 5.5 훈련 데이터 및 결과
    - 5.6 후속 발전: StyleGAN 시리즈
    - 5.7 PyTorch 구현
6. 전체 비교 및 요약

7. 핵심 수식 총정리
8. 참고문헌

---

1. 개요

1.1 GAN의 등장 배경

2014년 Ian Goodfellow가 제안한 **GAN(Generative Adversarial Network)**은 생성 모델 연구에 혁명을 일으켰다. 이후 폭발적인 연구가 이루어졌으며, Keras GAN 목록만 해도 약 300개 이상의 변형 모델이 존재한다.

> 📄 **논문**: Ian Goodfellow et al., "Generative Adversarial Nets", NeurIPS 2014

1.2 다루는 모델

모델	연도	저자	핵심 기여
Original GAN	2014	Ian Goodfellow (Google Brain)	GAN의 기본 프레임워크 제안
DCGAN	2015	Alec Radford (OpenAI)	CNN 기반 안정적 GAN 구조
ProGAN	2017	Tero Karras (NVIDIA)	점진적 해상도 증가로 1024×1024 생성

1.3 GAN vs 기존 생성 모델

이전에 배운 생성 모델들(VAE, Normalizing Flow, PixelRNN/CNN)과의 근본적인 차이:

측면	VAE/Flow/PixelCNN	GAN
확률 분포 학습	명시적 (Explicit)	암묵적 (Implicit)
학습 방식	최대 라이클리후드	적대적 훈련
목적 함수	단일 네트워크 최적화	두 네트워크의 Minimax Game
이론적 정당성	명확한 목적 함수	Nash Equilibrium

GAN의 핵심 철학:

- 확률 분포 $p(x)$를 명시적으로 학습하지 않는다
- **그럴싸한 이미지를 잘 생성하는 방법만** 학습한다
- 가장 큰 장점: **빠른 생성 속도**와 **높은 이미지 품질**

1.4 선수 지식

- **확률론**: 조건부 확률, 기댓값, KL divergence

- **최적화**: 경사하강법, Adam optimizer
- **딥러닝**: CNN, Transposed Convolution, Batch Normalization
- **이론**: Nash Equilibrium, Minimax 게임 이론

1.5 학습 목표

1. GAN의 Minimax 게임 구조와 수학적 원리 이해
2. 판별자와 생성자의 교대 훈련 메커니즘 이해
3. Mode Collapse 등 GAN의 고질적 문제와 해결 방법 학습
4. DCGAN, ProGAN 등 주요 변형 모델의 기법 습득
5. 실전 GAN 훈련을 위한 PyTorch 구현 능력 배양

---

2. Original GAN

2.1 핵심 아이디어: 위조지폐범 vs 경찰

GAN의 핵심은 **두 네트워크의 경쟁적 학습(Adversarial Training)**이다.

역할	네트워크	목표
위조지폐범	Generator G (생성자)	진짜 같은 이미지를 생성하여 판별자를 속인다
경찰	Discriminator D (판별자)	진짜 이미지와 가짜 이미지를 정확히 구분한다

서로 경쟁하며 발전하다 보면, 생성자는 **진짜와 구분할 수 없는 이미지**를 만들게 된다.

게임 이론적 관점

- **Two-player zero-sum game**: 한쪽이 이득을 보면 다른 쪽은 손해
- **Nash Equilibrium**: 양쪽 모두 더 이상 개선할 수 없는 균형점
- $p_g = p_{\text{data}}$일 때 균형 도달

2.2 모델 구조

```
[잠재 벡터 z] → [Generator G_φ] → 가짜 이미지 G(z) ┐
                                                    ├→ [Discrimi
                                 실제 이미지 x ─────────────┘
```

각 구성 요소의 역할:

Generator G_ϕ (생성자):

- 입력: 저차원 잠재 벡터 z (보통 100~512차원, $z \sim \mathcal{N}(0, I)$)

- 출력: 이미지 차원 (예: $128 \times 128 \times 3$)
- 오토인코더의 **디코더**에 해당
- 파라미터: ϕ
- 역할: $z \mapsto G_\phi(z)$, 저차원에서 고차원으로 매핑

Discriminator D_θ (판별자):

- 입력: 이미지 (진짜 또는 가짜, $128 \times 128 \times 3$)
- 출력: 진짜일 확률 (0~1 사이 스칼라, **시그모이드** 활성함수)
- **이진 분류기** (Binary Classifier)
- 파라미터: θ
- 역할: $x \mapsto D_\theta(x) \in [0, 1]$

핵심 포인트:

- 훈련이 끝나면 **판별자는 버린다** — 생성자만 있으면 이미지를 생성할 수 있다
- z에 랜덤한 값을 채워넣으면 진짜 같은 이미지가 생성된다
- VAE와 달리 인코더가 없음 — 이미지 → z 역변환 불가능

2.3 목적 함수 (Minimax Game)

GAN의 훈련 목표는 다음 **minimax 목적 함수**로 표현된다:

$$\min_\phi \max_\theta \; V(D_\theta, G_\phi) = \mathbb{E}_{x \sim p_{\text{data}}(x)}[\log D_\theta(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D_\theta(G_\phi(z)))]$$

- **판별자** D_θ: V를 **최대화(maximize)**하려 한다
- **생성자** G_ϕ: V를 **최소화(minimize)**하려 한다
- 같은 목적 함수를 놓고 양쪽이 반대 방향으로 당기는 **줄다리기**

왜 Minimax인가?

- $\max_\theta V$: 판별자가 V를 최대화
  - 진짜는 1, 가짜는 0으로 정확히 분류하면 V 최대
- $\min_\phi \max_\theta V$: 생성자가 그 최댓값을 최소화
  - 판별자가 속아서 가짜를 1로 판단하면 V 감소

수학적으로 이것은 **saddle point**를 찾는 문제이다.

2.4 판별자의 관점: Maximize

목적

$$\max_\theta \; V = \mathbb{E}_{x \sim p_{\text{data}}}[\log D_\theta(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D_\theta(G_\phi(z)))]$$

판별자는 파라미터 θ를 조절하여 V를 최대화한다.

각 항을 분석

첫 번째 항: $\mathbb{E}_{x\sim p_{\text{data}}}[\log D_\theta(x)]$

- x: **진짜 이미지**가 입력
- $D_\theta(x)$: 시그모이드 출력이므로 $0 \le D_\theta(x) \le 1$
- $\log$는 단조증가 함수이므로, $D_\theta(x) = 1$일 때 $\log D_\theta(x) = \log 1 = 0$ → **최대값**
- 즉, **진짜 이미지에 대해 D가 1(진짜)이라고 답하면** 이 항이 최대

두 번째 항: $\mathbb{E}_{z\sim p_z}[\log(1 - D_\theta(G_\phi(z)))]$

- $G_\phi(z)$: 생성자가 만든 **가짜 이미지**
- $D_\theta(G_\phi(z))$: 가짜 이미지를 판별자에 넣은 결과
- $1 - D_\theta(G_\phi(z))$: D가 0을 출력하면 $1 - 0 = 1$ → $\log 1 = 0$ → **최대값**
- 즉, **가짜 이미지에 대해 D가 0(가짜)이라고 답하면** 이 항이 최대

결론

$$\text{판별자의 목표: 진짜는 } 1\text{, 가짜는 } 0\text{으로 정확히 분류}$$

이것은 당연한 결과이다 — 판별자가 완벽히 분류하면 V가 최대가 된다.

손실 함수(Loss)로 변환

실제 훈련에서는 최대화가 아닌 **최소화**를 수행하므로, 부호를 반전한다:

$$\mathcal{L}_D = -\mathbb{E}_{x\sim p_{\text{data}}}[\log D_\theta(x)] - \mathbb{E}_{z\sim p_z}[\log(1 - D_\theta(G_\phi(z)))]$$

> $\max V$**는** $\min(-V)$**와 동일하다.**

Binary Cross-Entropy와의 관계

이것은 **Binary Cross-Entropy (BCE)** loss와 동일하다:

$$\text{BCE}(y, \hat{y}) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

- 진짜 이미지: $y = 1$, $\hat{y} = D(x)$ → $-\log D(x)$
- 가짜 이미지: $y = 0$, $\hat{y} = D(G(z))$ → $-\log(1 - D(G(z)))$

2.5 생성자의 관점: Minimize

목적

생성자 입장에서 첫 번째 항 $\mathbb{E}[\log D_\theta(x)]$은 **진짜 이미지에 대한 것**이므로 생성자의 파라미터 ϕ와 무관하다. 따라서 무시하고, 두 번째 항만 고려한다:

$$\min_\phi \ \mathbb{E}_{z\sim p_z}[\log(1 - D_\theta(G_\phi(z)))]$$

분석

- 생성자가 만든 가짜 이미지 $G_\phi(z)$를 판별자에 넣었을 때
- **판별자가 속아서 $D_\theta(G_\phi(z)) = 1$이라고 답하면**:
  - $\log(1-1) = \log 0 \to -\infty$ → **최소값** 달성!
- 즉, **판별자를 속이면** 생성자의 목표를 달성

손실 함수

$$\mathcal{L}_G = \mathbb{E}_{z\sim p_z}[\log(1 - D_\theta(G_\phi(z)))]$$

기대값은 여러 샘플에 대한 평균이다. 하나의 샘플에 대해서는:

$$\ell_G = \log(1 - D_\theta(G_\phi(z)))$$

2.6 Non-Saturating Loss (실전 트릭)

문제점: 초기 훈련의 그래디언트 소실

원래 생성자 손실 $\log(1 - D_\theta(G_\phi(z)))$의 그래프를 그려보자.

$D_\theta(G_\phi(z))$를 a로 놓으면 ($0 \le a \le 1$):

$$f(a) = \log(1-a)$$

```
Loss
 ↑
 0 ──────────────────────┐
                          \
                           \ 기울기  작음
                            \
                             \
-∞                            \
 └──────────────────────────→ D(G(z))
 0                            1
```

문제: 훈련 초기에 생성자가 만드는 이미지는 품질이 매우 낮다. 판별자가 쉽게 가짜임을 알아채므로 $D(G(z)) \approx 0$이다.

- $a \approx 0$ 영역에서 $f(a) = \log(1-a)$의 기울기가 **매우 완만**

$$\frac{df}{da} = \frac{-1}{1-a} \approx -1 \quad (a \approx 0\text{일 때})$$

- 그래디언트가 작아서 **생성자의 가중치가 거의 갱신되지 않음**
- 훈련이 극도로 느림 → **Vanishing Gradient** 문제

해결: 목적 함수 변환

$0 \leq a \leq 1$ 범위에서:

$$\min_{\phi} \ \log(1 - D(G(z))) \quad \Longleftrightarrow \quad \max_{\phi} \ \log D(G(z))$$

두 식은 **동일한 최적점**을 가진다 (출력이 0~1 사이라는 제약 하에).

다시 최소화 형태로 변환하면:

$$\boxed{\mathcal{L}_G^{\text{NS}} = -\mathbb{E}_{z \sim p_z}[\log D_\theta(G_\phi(z))]}$$

이것이 **Non-Saturating (NS) Loss**이다.

새로운 손실의 그래프

$g(a) = -\log(a)$의 그래프:

```
Loss
 ↑
 ∞
 \
  \ 기울기 매우 큼!
   \
    \
     \
      \
       \──────────────────── 0
└──────────────────────────────→ D(G(z))
0                              1
```

개선 효과: $a \approx 0$ 영역(초기 훈련)에서 기울기가 **매우 가파름**

$$\frac{dg}{da} = \frac{-1}{a} \to -\infty \quad (a \to 0\text{일 때})$$

→ 큰 그래디언트 → 빠른 학습

정리: 두 손실의 비교

	원래 손실	**Non-Saturating 손실**
식	$\log(1 - D(G(z)))$	$-\log D(G(z))$
초기 그래디언트	작음 (학습 느림)	큼 (학습 빠름)
최적점	동일	동일
실제 사용	이론적 분석용	**실전 훈련에 사용**

> 💡 Ian Goodfellow도 원래 손실로는 훈련이 잘 안 돼서 NS Loss를 사용했다고 밝혔다.

2.7 전체 손실 함수 정리

판별자 손실

$$\boxed{\mathcal{L}_D = -\frac{1}{m}\sum_{i=1}^{m}\log D_\theta(x^{(i)}) - \frac{1}{m}\sum_{i=1}^{m}\log(1 - D_\theta(G_\phi(z^{(i)})))}$$

- $x^{(i)}$: 실제 데이터에서 샘플링한 진짜 이미지
- $z^{(i)}$: $\mathcal{N}(0, I)$에서 샘플링한 잠재 벡터
- m: 배치 크기

PyTorch 구현:

```
d_loss_real = F.binary_cross_entropy(D(real_images), torch.ones_like
d_loss_fake = F.binary_cross_entropy(D(fake_images.detach()), torch
d_loss = d_loss_real + d_loss_fake
```

생성자 손실 (Non-Saturating)

$$\boxed{\mathcal{L}_G = -\frac{1}{m}\sum_{i=1}^{m}\log D_\theta(G_\phi(z^{(i)}))}$$

PyTorch 구현:

```
g_loss = F.binary_cross_entropy(D(fake_images), torch.ones_like(D(fa
```

2.8 교대 훈련 (Alternating Training)

GAN은 두 네트워크를 **동시에 훈련할 수 없다**. 번갈아가며 한쪽씩 훈련한다:

Step 1: 판별자 훈련 (생성자 가중치 고정)

```
진짜 이미지 x (레이블 = 1) ──→ [D_θ] ──→ Loss 계산
가짜 이미지 G(z) (레이블 = 0) ──→ [D_θ] ──→ Loss 계산
                                        ↓
                                 역전파: θ만 업데이트
                                 [G_φ는 🔒 고정]
```

1. 실제 데이터에서 배치 크기만큼 (예: 128장) 진짜 이미지 샘플링 → 레이블 **1**
2. z를 샘플링하여 생성자로 가짜 이미지 128장 생성 → 레이블 **0**
3. 두 배치를 판별자에 넣고 $\mathcal{L}_D$ 계산
4. 역전파하여 θ**(판별자 가중치)만 업데이트**

Step 2: 생성자 훈련 (판별자 가중치 고정)

```
z → [G_φ] → 가짜 이미지 (레이블 = 1⚠️) → [D_θ 🔒] → Loss 계산
                                                        ↓
                                                 역전파: φ만 업데이트
```

1. z를 샘플링하여 가짜 이미지 128장 생성
2. **레이블을 1(진짜)로 부여** ← 핵심 포인트!
3. 판별자를 통과시켜 $\mathcal{L}_G$ 계산
4. 역전파하여 ϕ**(생성자 가중치)만 업데이트**, 판별자는 고정

> ⚠️ **주의**: 생성자 훈련 시 가짜 이미지에 레이블 **1(진짜)**을 부여해야 한다. 이래야 판별자가 "진짜"라고 판단하는 방향으로 생성자가 학습된다.

교대 훈련 의사코드

```
for epoch in range(num_epochs):
    for each batch:
        # --- Step 1: 판별자 훈련 ---
        x_real = sample_from_dataset(batch_size)   # 진짜 이미지
        z = sample_noise(batch_size, z_dim)         # 잠재 벡터
        x_fake = G(z)                               # 가짜 이미지 생

        L_D_real = -mean(log(D(x_real)))            # 진짜에 대한 손
        L_D_fake = -mean(log(1 - D(x_fake.detach())))  # 가짜에 대한
        L_D = L_D_real + L_D_fake

        update θ using ▽_θ L_D                       # 판별자만 업데

        # --- Step 2: 생성자 훈련 ---
        z = sample_noise(batch_size, z_dim)
        x_fake = G(z)

        L_G = -mean(log(D(x_fake)))                 # NS Loss
        update φ using ▽_φ L_G                       # 생성자만 업데
```

판별자 vs 생성자 업데이트 비율

- 원래 논문: 판별자 1회 업데이트당 생성자 1회 업데이트
- 실전에서는 판별자를 더 자주 업데이트 (예: D:G = 5:1)
    - 판별자가 너무 강하면: 생성자가 학습 못함

    - 판별자가 너무 약하면: 생성자가 무의미한 이미지 생성

균형 유지가 핵심!

2.9 GAN의 이론적 분석: 최적 판별자

고정된 G에 대한 최적 판별자

G가 고정되어 있을 때, 목적 함수 $V(D, G)$를 D에 대해 최대화하면:

$$V = \int_x p_{\text{data}}(x) \log D(x)\, dx + \int_x p_g(x) \log(1 - D(x))\, dx$$

여기서 $p_g(x)$는 생성자가 만들어내는 이미지의 분포이다.

각 x에 대해 독립적으로 최대화할 수 있다. 피적분함수를 $D(x) = y$에 대해 미분하면:

$$f(y) = a \log y + b \log(1 - y)$$

여기서 $a = p_{\text{data}}(x)$, $b = p_g(x)$이다.

최적점을 찾기 위해 미분:

$$\frac{df}{dy} = \frac{a}{y} - \frac{b}{1-y} = 0$$

$$\frac{a}{y} = \frac{b}{1-y}$$

$$a(1-y) = by$$

$$a - ay = by$$

$$a = y(a+b)$$

$$\boxed{D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}}$$

> **해석**: 최적 판별자는 진짜 데이터 분포와 생성 데이터 분포의 비율로 결정된다.

균형 상태에서의 판별자

훈련이 완벽히 수렴하여 $p_g = p_{\text{data}}$가 되면:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_{\text{data}}(x)} = \frac{1}{2}$$

모든 이미지에 대해 $D^* = \frac{1}{2}$ → **진짜인지 가짜인지 전혀 구분 못함** → 훈련 완료

생성자의 최적화와 JS Divergence

최적 판별자 D^*를 $V(D, G)$에 대입하면:

$$C(G) = \max_D V(D, G) = V(D^*, G)$$

이를 전개하면:

$$C(G) = -\log 4 + 2 \cdot \mathrm{JS}(p_{\text{data}} \| p_g)$$

여기서 JS는 **Jensen-Shannon Divergence**이다:

$$\mathrm{JS}(p \| q) = \frac{1}{2} D_{KL}(p \| m) + \frac{1}{2} D_{KL}(q \| m), \quad m = \frac{p+q}{2}$$

따라서 생성자를 최적화하는 것은 **JS divergence를 최소화**하는 것과 동일하다!

2.10 GAN의 문제점

(1) Mode Collapse (모드 붕괴)

GAN은 확률 분포를 학습하지 않고 **판별자를 속이기만 하면 된다**.

- MNIST 예시: 0~9까지 다양한 숫자를 생성해야 하지만
- 생성자가 "5"를 만들어봤더니 판별자가 잘 속는다면?
- → 생성자는 **계속 5만 생성** → 다른 숫자는 만들지 않음
- 특정 mode에만 집중하여 **다양성이 사라지는 현상**

이것이 **Mode Collapse**이다. 손실을 줄이는 데는 성공하지만 다양한 이미지를 만들지 못한다.

수학적 설명:

데이터 분포가 multimodal: $p_{\text{data}} = \sum_{i=1}^{K} w_i p_i(x)$

생성 분포가 single mode만 커버: $p_g \approx p_1(x)$

- JS divergence는 감소하지만 (단일 모드에서는 잘 맞음)
- 전체 분포를 커버하지 못함

해결 방법:

- Minibatch Discrimination (ProGAN에서 사용)
- Unrolled GAN
- Spectral Normalization
- Multiple discriminators

(2) 훈련 불안정성

- **하이퍼파라미터 민감도**: learning rate 등을 조금만 잘못 설정하면 20,000회 반복해도 노이즈만 출력
- **초기값 민감도**: 가중치 초기값에 따라 결과가 크게 달라짐
- **수렴의 어려움**: 판별자와 생성자의 균형이 무너지면 훈련 실패
- 잘 만들어진 모델의 하이퍼파라미터를 조금만 바꿔도 전혀 동작하지 않는 경우 빈번

원인:

- Minimax 게임의 saddle point는 local minimum/maximum과 다름
- Gradient descent는 saddle point에서 진동하거나 발산할 수 있음

(3) Gradient Vanishing

판별자가 너무 강해지면:

- $D(x) \approx 1$, $D(G(z)) \approx 0$
- $\log(1 - D(G(z))) \approx 0$ → gradient ≈ 0
- 생성자가 학습 멈춤

(4) 평가 지표의 부재

- VAE: ELBO로 정량적 평가 가능
- GAN: likelihood를 모르므로 정량적 평가 어려움
- IS, FID 등은 간접적 지표일 뿐

2.11 PyTorch 구현

완전한 Vanilla GAN 구현

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import numpy as np
import matplotlib.pyplot as plt

# Hyperparameters
latent_dim = 100
image_shape = (1, 28, 28)  # MNIST
batch_size = 128
num_epochs = 100
lr = 0.0002
device = 'cuda' if torch.cuda.is_available() else 'cpu'


class Generator(nn.Module):
    """
    Generator network for Vanilla GAN

    Args:
        latent_dim: Dimension of latent vector z
        img_shape: Shape of output image (C, H, W)
    """
    def __init__(self, latent_dim, img_shape):
        super(Generator, self).__init__()
        self.img_shape = img_shape

        def block(in_features, out_features, normalize=True):
            layers = [nn.Linear(in_features, out_features)]
            if normalize:
                layers.append(nn.BatchNorm1d(out_features))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers

        self.model = nn.Sequential(
            *block(latent_dim, 128, normalize=False),
            *block(128, 256),
            *block(256, 512),
            *block(512, 1024),
            nn.Linear(1024, int(np.prod(img_shape))),
            nn.Tanh()  # Output in [-1, 1]
        )

    def forward(self, z):
        """
        Args:
            z: Latent vector of shape (B, latent_dim)

        Returns:
```

```
            Generated images of shape (B, C, H, W)
        """
        img = self.model(z)
        img = img.view(img.size(0), *self.img_shape)
        return img


class Discriminator(nn.Module):
    """
    Discriminator network for Vanilla GAN

    Args:
        img_shape: Shape of input image (C, H, W)
    """
    def __init__(self, img_shape):
        super(Discriminator, self).__init__()

        self.model = nn.Sequential(
            nn.Linear(int(np.prod(img_shape)), 512),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Linear(256, 1),
            nn.Sigmoid()  # Output probability in [0, 1]
        )

    def forward(self, img):
        """
        Args:
            img: Input images of shape (B, C, H, W)

        Returns:
            Probability of being real: (B, 1)
        """
        img_flat = img.view(img.size(0), -1)
        validity = self.model(img_flat)
        return validity


def train_gan(generator, discriminator, train_loader, num_epochs, de
    """
    Train GAN with alternating updates

    Args:
        generator: Generator model
        discriminator: Discriminator model
        train_loader: DataLoader for training data
        num_epochs: Number of training epochs
        device: Device to train on
    """
    # Optimizers
    optimizer_G = optim.Adam(generator.parameters(), lr=lr, betas=(
```

```
optimizer_G = optim.Adam(generator.parameters(), lr=lr, betas=(
optimizer_D = optim.Adam(discriminator.parameters(), lr=lr, beta

# Loss function
adversarial_loss = nn.BCELoss()

for epoch in range(num_epochs):
    for i, (imgs, _) in enumerate(train_loader):
        batch_size = imgs.shape[0]

        # Ground truth labels
        real_labels = torch.ones(batch_size, 1).to(device)
        fake_labels = torch.zeros(batch_size, 1).to(device)

        # Real images
        real_imgs = imgs.to(device)

        # -----------------
        # Train Discriminator
        # -----------------
        optimizer_D.zero_grad()

        # Loss on real images
        real_pred = discriminator(real_imgs)
        d_loss_real = adversarial_loss(real_pred, real_labels)

        # Generate fake images
        z = torch.randn(batch_size, latent_dim).to(device)
        fake_imgs = generator(z)

        # Loss on fake images
        fake_pred = discriminator(fake_imgs.detach())  # Detach
        d_loss_fake = adversarial_loss(fake_pred, fake_labels)

        # Total discriminator loss
        d_loss = d_loss_real + d_loss_fake
        d_loss.backward()
        optimizer_D.step()

        # -----------------
        # Train Generator
        # -----------------
        optimizer_G.zero_grad()

        # Generate new fake images
        z = torch.randn(batch_size, latent_dim).to(device)
        gen_imgs = generator(z)

        # Generator loss (fool discriminator)
        gen_pred = discriminator(gen_imgs)
        g_loss = adversarial_loss(gen_pred, real_labels)  # Wan
```

```
            g_loss.backward()
            optimizer_G.step()

            # -----------------
            # Logging
            # -----------------
            if i % 100 == 0:
                print(f"[Epoch {epoch}/{num_epochs}] [Batch {i}/{le
                      f"[D loss: {d_loss.item():.4f}] [G loss: {g_l

        # Save generated images every epoch
        if epoch % 10 == 0:
            save_generated_images(generator, epoch, device)


def save_generated_images(generator, epoch, device, num_images=25):
    """Generate and save sample images"""
    generator.eval()
    with torch.no_grad():
        z = torch.randn(num_images, latent_dim).to(device)
        gen_imgs = generator(z)
        gen_imgs = gen_imgs.cpu().numpy()

        # Plot
        fig, axes = plt.subplots(5, 5, figsize=(10, 10))
        for i, ax in enumerate(axes.flat):
            ax.imshow(gen_imgs[i, 0], cmap='gray')
            ax.axis('off')
        plt.savefig(f'gan_epoch_{epoch}.png')
        plt.close()
    generator.train()


# Example usage
if __name__ == "__main__":
    # Load MNIST dataset
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize([0.5], [0.5])  # Normalize to [-1, 1]
    ])

    mnist = datasets.MNIST(root='./data', train=True, download=True,
    train_loader = DataLoader(mnist, batch_size=batch_size, shuffle=

    # Initialize models
    generator = Generator(latent_dim, image_shape).to(device)
    discriminator = Discriminator(image_shape).to(device)

    # Train
    train_gan(generator, discriminator, train_loader, num_epochs, de

    # Generate samples after training
```

```
generator.eval()
with torch.no_grad():
    z = torch.randn(16, latent_dim).to(device)
    fake_images = generator(z)
```

주요 구현 포인트

1. **LeakyReLU vs ReLU**:
   - Discriminator에서 LeakyReLU 사용 (negative slope = 0.2)
   - 음수 gradient도 흘려보내 학습 안정화
2. **Batch Normalization**:
   - Generator에서 사용 (첫 층 제외)
   - Discriminator에서는 사용하지 않는 것이 일반적
3. **Optimizer 설정**:
   - Adam optimizer, lr=0.0002
   - Beta1=0.5 (기본값 0.9 대신)
   - 더 빠른 수렴과 안정성
4. **detach() 사용**:
   - Discriminator 훈련 시 `fake_imgs.detach()`
   - Generator로 gradient가 흐르지 않도록 차단

---

3. 생성 모델의 성능 평가 지표

3.1 Inception Score (IS)

사전 훈련된 Inception V3 모델을 사용하여 평가한다.

평가 기준 두 가지

① **이미지 품질 (Quality)**: 생성된 이미지를 Inception V3에 넣었을 때, **특정 클래스에 대한 확률이 높아야** 한다.

- 조건부 분포 $p(y \mid x)$가 **낮은 엔트로피** (뾰족한 분포)
- "이건 확실히 고양이다"처럼 명확하게 분류되어야 좋은 이미지
- 수식: $H(p(y \mid x)) = -\sum_y p(y \mid x) \log p(y \mid x)$ 작아야 함

② **다양성 (Diversity)**: 생성된 이미지 전체가 **다양한 클래스에 고루 분포**해야 한다.

- 주변 분포 $p(y)$가 **높은 엔트로피** (균일한 분포)
- 특정 클래스에 치우치지 않아야 함
- 수식: $H(p(y)) = -\sum_y p(y) \log p(y)$ 커야 함

수식

$$\mathrm{IS} = \exp\left(\mathbb{E}_{x \sim p_g}\left[D_{KL}(p(y \mid x) \| p(y))\right]\right)$$

여기서:

$$D_{KL}(p(y \mid x) \| p(y)) = \sum_y p(y \mid x) \log \frac{p(y \mid x)}{p(y)}$$

직관적 이해

- $p(y \mid x)$: 뾰족 (낮은 엔트로피) - 좋은 품질
- $p(y)$: 균일 (높은 엔트로피) - 높은 다양성
- KL divergence: 두 분포의 차이가 클수록 IS 증가

범위

- **IS가 높을수록 좋다**
- ImageNet (1000 classes): 좋은 모델 IS ≈ 10~20
- CIFAR-10 (10 classes): 좋은 모델 IS ≈ 6~8

한계

- 사전 훈련된 Inception V3에 의존
- ImageNet 외 도메인에서는 부정확할 수 있음
- Intra-class diversity는 측정 못함

3.2 Fréchet Inception Distance (FID)

역시 **Inception V3**를 사용한다.

아이디어

실제 데이터와 생성된 데이터를 Inception V3의 중간 레이어에 통과시켜 **피처 벡터**를 추출한 뒤, 두 분포의 거리를 측정한다.

수식

$$\mathrm{FID} = \|\mu_r - \mu_g\|^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\right)$$

- μ_r, Σ_r: 실제 데이터의 피처 벡터 평균, 공분산
- μ_g, Σ_g: 생성된 데이터의 피처 벡터 평균, 공분산
- Tr: Trace (대각합)

직관적 이해

두 다변량 가우시안 분포의 **Fréchet distance** (또는 Wasserstein-2 distance for Gaussians)

1. **평균 차이**: $\|\mu_r - \mu_g\|^2$
    - 생성 이미지의 평균적 특성이 실제와 얼마나 다른가
2. **공분산 차이**: $\mathrm{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2})$
    - 생성 이미지의 다양성과 상관관계가 실제와 얼마나 다른가

범위

- **FID가 낮을수록 좋다** (두 분포가 가까움)
- CIFAR-10: FID < 10 (좋음), FID < 5 (매우 좋음)
- ImageNet: FID < 20 (좋음), FID < 10 (매우 좋음)

IS vs FID

측면	Inception Score	FID
측정 대상	생성 이미지만	생성 + 실제 이미지
다양성	Mode collapse 감지 어려움	더 잘 감지
실제 데이터 필요	불필요	필요
일반적 선호	덜 선호	**더 선호**

3.3 기타 지표

지표	의미	좋은 방향	계산 방법
Precision	생성된 이미지가 실제 이미지와 얼마나 비슷한가	높을수록 좋음	k-NN 기반
Recall	생성된 이미지가 얼마나 다양한가	높을수록 좋음	k-NN 기반
KL Divergence	실제 분포와 생성 분포의 차이	낮을수록 좋음	$D_{KL}(p\|\|q)$
Jensen-Shannon Divergence	KL의 대칭 버전	낮을수록 좋음	JS = (KL(p||m) + KL(q||m))/2
MMD	Maximum Mean Discrepancy	낮을수록 좋음	Kernel-based

가장 널리 사용되는 것: **IS**, **FID**, **Precision/Recall**

---

4. DCGAN (Deep Convolutional GAN)

> 📄 **논문**: Alec Radford et al., "Unsupervised Representation Learning with Deep Convolutional GANs", ICLR 2016
>
> 저자 Alec Radford는 이후 **CLIP**, **GPT**, **DALL-E**의 공동 저자이기도 하다 (OpenAI).

4.1 핵심 아이디어

Original GAN의 얕은 Fully Connected 구조를 **Deep CNN 구조**로 교체한 것이다.

혁신:

- 생성자와 판별자 모두 **깊은 CNN** 사용
- Batch Normalization 도입 (2015년 당시 막 나온 기술)
- 이미지 해상도: $64 \times 64 \times 3$ (컬러)
- **훈련 안정성 대폭 향상**

4.2 네트워크 구조

Generator (100차원 → 64×64×3)

```
z (100차원) → [Project & Reshape: 4×4×1024]
            → [TransConv 512: 8×8] → BN → ReLU
            → [TransConv 256: 16×16] → BN → ReLU
            → [TransConv 128: 32×32] → BN → ReLU
            → [TransConv 3: 64×64] → Tanh
```

- **Transposed Convolution** (역컨볼루션)으로 해상도를 점진적으로 확대
- 중간층: Batch Normalization + ReLU
- 출력층: Tanh ([-1, 1] 범위)

Discriminator (64×64×3 → 1)

```
이미지 (64×64×3) → [Conv 64: 32×32] → LeakyReLU(0.2)
                 → [Conv 128: 16×16] → BN → LeakyReLU(0.2)
                 → [Conv 256: 8×8] → BN → LeakyReLU(0.2)
                 → [Conv 512: 4×4] → BN → LeakyReLU(0.2)
                 → [Flatten → Dense 1] → Sigmoid
```

- 일반 Convolution으로 해상도를 점진적으로 축소
- 최종: Sigmoid로 이진 분류

4.3 잠재 공간의 산술 연산

GAN은 잠재 공간(z space)을 명시적으로 학습하지 않지만, 흥미롭게도 잠재 벡터에 ** 방향성(direction)**이 존재한다.

벡터 연산으로 속성 조작

예시 1: "웃음" 벡터 발견

1. 다양한 이미지를 생성하여 "웃는 여자"를 만들어낸 잠재 벡터들 z_1, z_2, z_3 수집
2. 평균: $\bar{z}_{\text{smiling_woman}} = \frac{1}{N}\sum z_i$
3. 마찬가지로 $\bar{z}_{\text{neutral_woman}}$, $\bar{z}_{\text{neutral_man}}$ 계산

그러면:

$$\bar{z}_{\text{smiling_man}} = \bar{z}_{\text{smiling_woman}} - \bar{z}_{\text{neutral_woman}} + \bar{z}_{\text{neutral_man}}$$

$$\text{웃는 남자} = \text{웃는 여자} - \text{무표정한 여자} + \text{무표정한 남자}$$

풀이: $\bar{z}_{\text{smiling_woman}} - \bar{z}_{\text{neutral_woman}}$은 "웃음" 성분만 추출한 것이다 (같은 여자인데 웃음과 무표정의 차이). 이것을 무표정한 남자에 더하면 **웃는 남자**가 된다.

예시 2: "안경" 벡터 발견

$$\text{안경 낀 여자} = \text{안경 낀 남자} - \text{안경 안 낀 남자} + \text{안경 안 낀 여자}$$

같은 원리로, "안경" 성분을 추출하여 다른 얼굴에 적용 가능.

> **핵심**: Word2Vec에서 "King - Man + Woman = Queen"과 동일한 원리가 이미지 잠재 공간에서도 작동한다.

주의: 픽셀 공간 vs 잠재 공간

- **잠재 공간에서 연산** 후 이미지 생성 → 의미 있는 결과
- **픽셀 공간에서 직접 더하기/빼기** → 엉망인 이미지 (단순 블렌딩)

선형 보간 (Linear Interpolation)

z_1과 z_2 사이를 선형적으로 보간하면:

$$z_t = (1-t)\cdot z_1 + t\cdot z_2, \quad t \in [0,1]$$

각 z_t로 이미지를 생성하면, 한 사람에서 다른 사람으로 **부드럽게 변화**하는 과정을 볼 수 있다.

이는 잠재 공간이 **연속적이고 의미 있는 구조**를 갖고 있음을 보여준다.

4.4 훈련 가이드라인

DCGAN 논문에서 제시한 **안정적 훈련을 위한 지침**:

1. **Pooling layer 제거**:
    - Spatial pooling (MaxPool, AvgPool) 사용하지 않음
    - Generator: Transposed Convolution으로 upsampling

    - Discriminator: Strided Convolution으로 downsampling

2. **Batch Normalization 사용**:
    - Generator: 모든 층에 BN (출력층 제외)
    - Discriminator: 모든 층에 BN (입력층 제외)

3. **Fully Connected hidden layer 제거**:
    - Deep architecture로 대체

4. **활성 함수**:
    - Generator 출력층: Tanh
    - Generator 중간층: ReLU
    - Discriminator 모든 층: LeakyReLU (slope=0.2)

5. **Optimizer**:
    - Adam: lr=0.0002, beta1=0.5

4.5 PyTorch 구현

```python
import torch
import torch.nn as nn

class DCGANGenerator(nn.Module):
    """
    DCGAN Generator

    Args:
        latent_dim: Dimension of latent vector z
        img_channels: Number of image channels (3 for RGB)
        feature_maps: Base number of feature maps (default 64)
    """
    def __init__(self, latent_dim=100, img_channels=3, feature_maps=
        super(DCGANGenerator, self).__init__()

        self.init_size = 4  # Initial spatial size
        self.l1 = nn.Sequential(
            nn.Linear(latent_dim, feature_maps * 8 * self.init_size
        )

        self.conv_blocks = nn.Sequential(
            # 4x4 -> 8x8
            nn.ConvTranspose2d(feature_maps * 8, feature_maps * 4, 4
            nn.BatchNorm2d(feature_maps * 4),
            nn.ReLU(True),

            # 8x8 -> 16x16
            nn.ConvTranspose2d(feature_maps * 4, feature_maps * 2, 4
            nn.BatchNorm2d(feature_maps * 2),
            nn.ReLU(True),

            # 16x16 -> 32x32
            nn.ConvTranspose2d(feature_maps * 2, feature_maps, 4, 2,
            nn.BatchNorm2d(feature_maps),
            nn.ReLU(True),

            # 32x32 -> 64x64
            nn.ConvTranspose2d(feature_maps, img_channels, 4, 2, 1,
            nn.Tanh()  # Output range [-1, 1]
        )

    def forward(self, z):
        """
        Args:
            z: Latent vector of shape (B, latent_dim)

        Returns:
            Generated images of shape (B, C, 64, 64)
        """
        out = self.l1(z)
        out = out.view(out.shape[0], -1, self.init_size, self.init_s
        img = self.conv_blocks(out)
```

```python
        return img


class DCGANDiscriminator(nn.Module):
    """
    DCGAN Discriminator

    Args:
        img_channels: Number of image channels (3 for RGB)
        feature_maps: Base number of feature maps (default 64)
    """
    def __init__(self, img_channels=3, feature_maps=64):
        super(DCGANDiscriminator, self).__init__()

        self.conv_blocks = nn.Sequential(
            # 64x64 -> 32x32
            nn.Conv2d(img_channels, feature_maps, 4, 2, 1, bias=Fal
            nn.LeakyReLU(0.2, inplace=True),

            # 32x32 -> 16x16
            nn.Conv2d(feature_maps, feature_maps * 2, 4, 2, 1, bias
            nn.BatchNorm2d(feature_maps * 2),
            nn.LeakyReLU(0.2, inplace=True),

            # 16x16 -> 8x8
            nn.Conv2d(feature_maps * 2, feature_maps * 4, 4, 2, 1, 
            nn.BatchNorm2d(feature_maps * 4),
            nn.LeakyReLU(0.2, inplace=True),

            # 8x8 -> 4x4
            nn.Conv2d(feature_maps * 4, feature_maps * 8, 4, 2, 1, 
            nn.BatchNorm2d(feature_maps * 8),
            nn.LeakyReLU(0.2, inplace=True),

            # 4x4 -> 1x1
            nn.Conv2d(feature_maps * 8, 1, 4, 1, 0, bias=False),
            nn.Sigmoid()
        )

    def forward(self, img):
        """
        Args:
            img: Input images of shape (B, C, 64, 64)

        Returns:
            Probability of being real: (B, 1, 1, 1)
        """
        validity = self.conv_blocks(img)
        return validity.view(-1, 1)


def weights_init(m):
```

```python
def weights_init(m):
    """
    Custom weights initialization for DCGAN
    """
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)


# Example usage
if __name__ == "__main__":
    latent_dim = 100
    device = 'cuda' if torch.cuda.is_available() else 'cpu'

    # Initialize
    generator = DCGANGenerator(latent_dim).to(device)
    discriminator = DCGANDiscriminator().to(device)

    # Apply weights initialization
    generator.apply(weights_init)
    discriminator.apply(weights_init)

    # Test forward pass
    z = torch.randn(4, latent_dim).to(device)
    fake_imgs = generator(z)
    print(f"Generated images shape: {fake_imgs.shape}")  # (4, 3, 64

    validity = discriminator(fake_imgs)
    print(f"Discriminator output shape: {validity.shape}")  # (4, 1,
```

주요 하이퍼파라미터

파라미터	권장 값	설명
latent_dim	100	잠재 벡터 차원
feature_maps	64	Base number of filters
learning_rate	0.0002	Adam optimizer
beta1	0.5	Adam beta1 (기본값 0.9 대신)
batch_size	64-128	배치 크기
num_epochs	50-200	훈련 에폭 수

---

5. ProGAN (Progressive Growing of GANs)

> 📄 **논문**: Tero Karras et al., "Progressive Growing of GANs for Improved Quality, Stability, and Variation", ICLR 2018
>
> 저자 Tero Karras는 NVIDIA 연구원으로, 이후 StyleGAN 시리즈(v1, v2, v3)도 개발. 매년 하나씩 발표하며 매번 놀라운 발전을 보여줌.

5.1 핵심 아이디어: 점진적 해상도 증가

기존 GAN의 문제:

- 고해상도 이미지(예: 1024×1024)를 **한 번에** 생성하려면 매우 어렵다
- 경사하강법으로 최적점을 찾을 때, 전체 landscape을 한 번에 보는 것과 같다
- 비유: 수학 공부를 할 때 갑자기 가장 어려운 문제집부터 시작하는 것

ProGAN의 해결책: **쉬운 것부터 점진적으로**

$$4 \times 4 \;\rightarrow\; 8 \times 8 \;\rightarrow\; 16 \times 16 \;\rightarrow\; 32 \times 32 \;\rightarrow\; \cdots \;\rightarrow\; 1024 \times 1024$$

1. 먼저 4×4 이미지를 생성하는 법을 학습
2. 어느 정도 학습되면 8×8 해상도로 확장
3. 이를 반복하여 1024×1024까지 도달

비유: 기초 문제집 → 중급 → 고급 → 최상급으로 단계적 학습

왜 효과적인가?

- 저해상도에서 global structure 학습 (얼굴의 대략적 배치)
- 고해상도로 갈수록 fine details 추가 (주름, 머리카락 텍스처)
- Curriculum learning: 쉬운 것부터 어려운 것으로

5.2 점진적 레이어 추가 방법

구조의 성장 과정

Phase 1: 4×4 생성

```
z → [G_4×4] → 4×4 이미지 (생성)
                                    → [D_4×4] → 진짜/가짜
      실제 데이터 (4×4로 다운스케일) ─┘
```

Phase 2: 8×8 생성 (레이어 추가)

```
z → [G_4×4] → [G_8×8 (새 레이어)] → 8×8 이미지
                                                → [D_8×8 (새 레
      실제 데이터 (8×8로 다운스케일) ──────────────┘
```

Phase N: 1024×1024 생성까지 반복

> **중요**: 이전 단계의 레이어 가중치가 **동결되지 않는다**. 새 레이어가 추가되어도 기존 레이어는 **계속 훈련**된다.

실제 구현에서는 뉴럴넷 구조를 처음부터 전체 구성해두고, 각 단계에서 **해당 레이어를 enable/disable**하는 방식으로 동작한다.

Smooth Transition (부드러운 전환)

레이어를 한 번에 추가하면 갑작스러운 변화로 훈련이 불안정해질 수 있다. 이를 방지하기 위해 **알파 블렌딩**을 사용한다.

16×16에서 32×32로 전환할 때:

$$\text{output} = (1 - \alpha) \cdot \text{upscale}(16 \times 16 \text{ output}) + \alpha \cdot (32 \times 32 \text{ output})$$

- $\alpha = 0$: 이전 해상도만 사용
- $\alpha \to 1$: 점진적으로 새 해상도 비중 증가 (linear ramp)
- $\alpha = 1$: 완전히 새 해상도로 전환

이렇게 하면 **급격한 변화 없이** 부드럽게 다음 해상도로 넘어간다.

학습 스케줄 예시

Phase	Resolution	Iterations	α schedule
1	4×4	200k	-
2	8×8	400k	0→1 (first 200k)
3	16×16	800k	0→1 (first 400k)
4	32×32	1.6M	0→1 (first 800k)
...	...	...	...

5.3 Mini-Batch Standard Deviation

문제: Mode Collapse

기존 GAN의 고질적 문제 — 생성자가 **비슷한 이미지만 반복 생성**

해결: 배치 내 표준편차 활용

핵심 관찰:

- **실제 이미지** 배치: 다양한 이미지들 → **큰 표준편차**
- **가짜 이미지** 배치 (mode collapse 시): 비슷한 이미지들 → **작은 표준편차**

방법:

1. 미니배치 내 이미지들의 feature map 표준편차 계산:

$$\sigma = \sqrt{\frac{1}{NHW}\sum_{n,h,w}(f_{nchw} - \mu_c)^2 + \epsilon}$$

여기서:

- f_{nchw}: 배치 내 n번째 이미지의 채널 c, 위치 (h, w)의 feature
- μ_c: 채널 c의 평균
- $\epsilon = 10^{-8}$ (수치 안정성)

2. 평균 표준편차를 모든 spatial location에 복제:

$$\bar{\sigma} = \frac{1}{C}\sum_{c=1}^{C}\sigma_c$$

3. 이를 추가 feature map으로 discriminator에 concatenate

```
# B = batch size, C = channels, H = height, W = width
batch_statistics = torch.std(features, dim=0).mean()  # Scalar
batch_statistics = batch_statistics.repeat(B, 1, H, W)  # Replicate
features = torch.cat([features, batch_statistics], dim=1)  # Concat
```

4. 표준편차가 작은 배치(= 다양성이 낮은)를 판별자가 **가짜로 판정**하도록 학습
5. 생성자는 판별자를 속이기 위해 **다양한 이미지를 생성**해야 함

직관적 예시

- MNIST에서 생성자가 "1"만 128장 생성 → 배치 표준편차 매우 작음 → 판별자가 가짜로 판정 → 큰 손실
- 0~9를 골고루 생성 → 배치 표준편차 큼 → 실제 데이터와 유사 → 작은 손실

5.4 기타 기법

Pixelwise Normalization

Batch Normalization 대신 사용. Generator에서만 적용.

$$\mathbf{b}_{x,y} = \frac{\mathbf{a}_{x,y}}{\sqrt{\frac{1}{C}\sum_{c=0}^{C-1}(a_{c,x,y})^2 + \epsilon}}$$

- 각 픽셀 위치의 feature 벡터를 unit length로 정규화
- BN처럼 배치 통계에 의존하지 않음
- Generator의 magnitude escalation 방지

Equalized Learning Rate

가중치를 초기화 시 $\mathcal{N}(0, 1)$로 하되, 런타임에 스케일링:

$$w_i = \frac{\hat{w}_i}{c}$$

여기서:

- $\hat{w}_i$: He initialization으로 초기화된 가중치
- $c = \sqrt{2/(\text{fan_in})}$

목적: 모든 층의 학습 속도를 동일하게 만듦

5.5 훈련 데이터 및 결과

데이터셋

- **CelebA-HQ**: 연예인 얼굴 고화질 데이터셋 (30,000장, 1024×1024)
- **LSUN**: Large-scale Scene Understanding — 카테고리별 대규모 이미지 데이터셋

카테고리	내용	이미지 수
Bridge	다리 사진	~800k
Bus	버스 사진	~500k
Car	자동차 사진	~3.8M
Church	교회 사진	~126k
Bedroom	침실 사진	~3M

> **중요**: ProGAN은 **카테고리별 전용 모델**을 따로 훈련해야 한다. 얼굴로 훈련한 모델은 얼굴만 생성 가능하고, 자동차로 훈련한 모델은 자동차만 생성 가능. 하나의 모델로 여러 카테고리를 생성하는 것은 불가. (조건부 GAN은 제외)

생성 결과

- **해상도**: 1024×1024
- 64×64에서 **16배 증가**한 해상도
- 처음 발표 시 "실제 데이터셋에서 가져온 거 아니냐"는 오해를 받을 정도로 사실적
- 훈련 시간: 약 **4-6일** (고성능 GPU 8개 기준)

성능 지표

- FID (CelebA-HQ 1024×1024): **7.3**
- 이전 SOTA 대비 큰 폭 개선

5.6 후속 발전: StyleGAN 시리즈

모델	연도	핵심 특징	주요 개선
StyleGAN v1	2018	Style-based generator	AdaIN으로 스타일 주입, Mapping network
StyleGAN v2	2019	점진적 성장 제거	Weight demodulation, Path length regularization
StyleGAN v3	2021	Alias-free	위치 불변성 개선, 애니메이션 품질 향상

> 각 버전이 이전 방법을 과감하게 버리고 완전히 새로운 접근을 도입한 것이 특징

StyleGAN의 혁신

Mapping Network: $z \to w$

- 512차원 z → 8층 FC → 512차원 w
- w는 disentangled representation (독립적 속성 제어)

AdaIN (Adaptive Instance Normalization):

$$\mathrm{AdaIN}(x_i, y) = y_{s,i}\frac{x_i - \mu(x_i)}{\sigma(x_i)} + y_{b,i}$$

- $y = (y_s, y_b)$: style vector (from w)
- 각 층마다 다른 style 주입 → 다양한 속성 제어

5.7 PyTorch 구현

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class MinibatchStdDev(nn.Module):
    """
    Minibatch standard deviation layer for ProGAN
    """
    def __init__(self):
        super(MinibatchStdDev, self).__init__()

    def forward(self, x):
        """
        Args:
            x: Input features of shape (B, C, H, W)

        Returns:
            Features with minibatch stddev appended: (B, C+1, H, W)
        """
        batch_statistics = torch.std(x, dim=0).mean()
        batch_statistics = batch_statistics.repeat(x.shape[0], 1, x
        return torch.cat([x, batch_statistics], dim=1)


class PixelNormalization(nn.Module):
    """
    Pixelwise feature vector normalization
    """
    def __init__(self, epsilon=1e-8):
        super(PixelNormalization, self).__init__()
        self.epsilon = epsilon

    def forward(self, x):
        """
        Args:
            x: Input features of shape (B, C, H, W)

        Returns:
            Normalized features: (B, C, H, W)
        """
        return x / torch.sqrt(torch.mean(x**2, dim=1, keepdim=True)


class EqualizedConv2d(nn.Module):
    """
    Convolutional layer with equalized learning rate
    """
    def __init__(self, in_channels, out_channels, kernel_size, stri
        super(EqualizedConv2d, self).__init__()

        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size
        self.bias = nn.Parameter(torch.zeros(out_channels))
```

```
        # Equalized learning rate
        self.scale = (2 / (in_channels * kernel_size * kernel_size)

        # Initialize with N(0,1)
        nn.init.normal_(self.conv.weight)

    def forward(self, x):
        return self.conv(x * self.scale) + self.bias.view(1, -1, 1,


class ProGANGeneratorBlock(nn.Module):
    """
    Generator block for ProGAN
    """
    def __init__(self, in_channels, out_channels, initial_block=Fal
        super(ProGANGeneratorBlock, self).__init__()
        self.initial_block = initial_block

        if initial_block:
            # First block: 1x1 -> 4x4
            self.conv1 = EqualizedConv2d(in_channels, out_channels,
            self.conv2 = EqualizedConv2d(out_channels, out_channels,
        else:
            # Other blocks: upsample then conv
            self.upsample = nn.Upsample(scale_factor=2, mode='neare
            self.conv1 = EqualizedConv2d(in_channels, out_channels,
            self.conv2 = EqualizedConv2d(out_channels, out_channels,

        self.pix_norm = PixelNormalization()
        self.leaky_relu = nn.LeakyReLU(0.2)

    def forward(self, x):
        if not self.initial_block:
            x = self.upsample(x)

        x = self.leaky_relu(self.conv1(x))
        x = self.pix_norm(x)
        x = self.leaky_relu(self.conv2(x))
        x = self.pix_norm(x)

        return x


class ProGANGenerator(nn.Module):
    """
    ProGAN Generator with progressive growing

    Args:
        latent_dim: Dimension of latent vector
        max_resolution: Maximum resolution (e.g., 1024)
        base_channels: Base number of channels (default 512)
```

```
        base_channels: Base number of channels (default 512)
    """
    def __init__(self, latent_dim=512, max_resolution=1024, base_cha
        super(ProGANGenerator, self).__init__()

        self.latent_dim = latent_dim
        self.max_resolution = max_resolution

        # Initial block (4x4)
        self.initial_block = ProGANGeneratorBlock(latent_dim, base_c

        # Progressive blocks
        self.blocks = nn.ModuleList()
        self.to_rgb = nn.ModuleList()

        resolution = 4
        in_channels = base_channels

        while resolution < max_resolution:
            resolution *= 2
            out_channels = max(base_channels // (resolution // 4), 1

            self.blocks.append(ProGANGeneratorBlock(in_channels, out
            self.to_rgb.append(EqualizedConv2d(out_channels, 3, 1, 1

            in_channels = out_channels

    def forward(self, z, alpha=1.0, steps=0):
        """
        Args:
            z: Latent vector (B, latent_dim)
            alpha: Blending factor for smooth transition
            steps: Current training step (0 = 4x4, 1 = 8x8, etc.)

        Returns:
            Generated images
        """
        # Initial block
        out = z.view(z.shape[0], -1, 1, 1)
        out = self.initial_block(out)

        if steps == 0:
            return self.to_rgb[0](out)

        # Progressive blocks
        for i in range(steps - 1):
            out = self.blocks[i](out)

        # Smooth transition
        out_old = self.to_rgb[steps - 1](out)
        out_old = F.interpolate(out_old, scale_factor=2, mode='near
```

```
        out_new = self.blocks[steps - 1](out)
        out_new = self.to_rgb[steps](out_new)

        return (1 - alpha) * out_old + alpha * out_new


class ProGANDiscriminatorBlock(nn.Module):
    """
    Discriminator block for ProGAN
    """
    def __init__(self, in_channels, out_channels):
        super(ProGANDiscriminatorBlock, self).__init__()

        self.conv1 = EqualizedConv2d(in_channels, out_channels, 3, 1
        self.conv2 = EqualizedConv2d(out_channels, out_channels, 3,
        self.leaky_relu = nn.LeakyReLU(0.2)
        self.downsample = nn.AvgPool2d(2)

    def forward(self, x):
        x = self.leaky_relu(self.conv1(x))
        x = self.leaky_relu(self.conv2(x))
        x = self.downsample(x)
        return x


class ProGANDiscriminator(nn.Module):
    """
    ProGAN Discriminator with progressive growing
    """
    def __init__(self, max_resolution=1024, base_channels=512):
        super(ProGANDiscriminator, self).__init__()

        self.max_resolution = max_resolution

        # Progressive blocks
        self.blocks = nn.ModuleList()
        self.from_rgb = nn.ModuleList()

        resolution = max_resolution
        out_channels = base_channels

        while resolution > 4:
            in_channels = max(out_channels // 2, 16)

            self.from_rgb.insert(0, EqualizedConv2d(3, in_channels,
            self.blocks.insert(0, ProGANDiscriminatorBlock(in_channe

            resolution //= 2
            out_channels = in_channels

        # Final block (4x4)
        self.minibatch_std = MinibatchStdDev()
```

```python
        self.final_conv = EqualizedConv2d(base_channels + 1, base_cl
        self.final_linear = nn.Sequential(
            nn.Flatten(),
            nn.Linear(base_channels * 4 * 4, base_channels),
            nn.LeakyReLU(0.2),
            nn.Linear(base_channels, 1)
        )

    def forward(self, x, alpha=1.0, steps=0):
        """
        Args:
            x: Input images
            alpha: Blending factor
            steps: Current training step

        Returns:
            Discriminator output (logits)
        """
        # Smooth transition
        if steps > 0:
            out_new = self.from_rgb[steps](x)
            out_old = F.avg_pool2d(x, 2)
            out_old = self.from_rgb[steps - 1](out_old)

            x = (1 - alpha) * out_old + alpha * out_new
            x = self.blocks[steps](x)
        else:
            x = self.from_rgb[0](x)

        # Progressive blocks
        for i in range(steps - 1, -1, -1):
            x = self.blocks[i](x)

        # Final layers
        x = self.minibatch_std(x)
        x = self.final_conv(x)
        x = self.final_linear(x)

        return x


# Example usage
if __name__ == "__main__":
    latent_dim = 512
    device = 'cuda' if torch.cuda.is_available() else 'cpu'

    # Initialize
    generator = ProGANGenerator(latent_dim, max_resolution=64).to(de
    discriminator = ProGANDiscriminator(max_resolution=64).to(devic

    # Test progressive growing
    z = torch.randn(4, latent_dim).to(device)
```

```
z = torch.randn(1, latent_dim).to(device)

# Step 0: 4x4
img_4x4 = generator(z, alpha=1.0, steps=0)
print(f"4x4: {img_4x4.shape}")

# Step 1: 8x8 (transitioning)
img_8x8 = generator(z, alpha=0.5, steps=1)
print(f"8x8 (α=0.5): {img_8x8.shape}")

# Discriminator
validity = discriminator(img_8x8, alpha=0.5, steps=1)
print(f"Discriminator output: {validity.shape}")
```

훈련 팁

1. **Progressive growing schedule**:
   - 각 해상도에서 충분히 훈련 (수십만 iterations)
   - α를 천천히 증가 (수만 iterations에 걸쳐)
2. **Learning rate**:
   - Generator: 0.001
   - Discriminator: 0.001
   - Adam: beta1=0, beta2=0.99
3. **Batch size**:
   - 고해상도로 갈수록 감소 (메모리 제약)
   - 1024×1024: batch size 16

---

6. 전체 비교 및 요약

6.1 GAN vs VAE vs Normalizing Flow

속성	GAN	VAE	Normalizing Flow
확률 분포 학습	✗ (Implicit)	○ (ELBO 근사)	○ (Exact likelihood)
학습 방식	적대적 훈련	최대 라이클리후드	최대 라이클리후드
생성 속도	매우 빠름	빠름	느림 (역변환 필요)
이미지 품질	매우 높음	보통 (blurry)	보통
다양성	Mode collapse 위험	좋음	좋음
훈련 안정성	불안정	안정	안정

잠재 공간	암묵적으로 학습	명시적으로 학습	가역 변환
손실 함수	Minimax game	ELBO = Recon + KL	$\log p(z) + \log \lvert \det J \rvert$
평가 지표	IS, FID (간접적)	ELBO (직접적)	NLL (직접적)

6.2 GAN 변형 모델 비교

모델	해상도	주요 기법	훈련 시간	안정성
Vanilla GAN	28×28	FC layers	짧음	매우 불안정
DCGAN	64×64	CNN, BN	중간	보통
ProGAN	1024×1024	Progressive growing	매우 길	좋음
StyleGAN	1024×1024	Style modulation	매우 길	매우 좋음

7. 핵심 수식 총정리

GAN 목적 함수

$$\min_G \max_D \; \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

판별자 손실

$$\mathcal{L}_D = -\mathbb{E}[\log D(x)] - \mathbb{E}[\log(1 - D(G(z)))]$$

생성자 손실 (Non-Saturating)

$$\mathcal{L}_G = -\mathbb{E}[\log D(G(z))]$$

최적 판별자

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

균형 상태

$$p_g = p_{\text{data}} \implies D^*(x) = \frac{1}{2} \quad \forall x$$

Inception Score

$$\text{IS} = \exp\left(\mathbb{E}_x \left[D_{KL}(p(y \mid x) \| p(y))\right]\right)$$

FID

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right)$$

ProGAN Smooth Transition

$$\text{output} = (1-\alpha)\cdot\text{upscale}(x_{\text{low}}) + \alpha\cdot x_{\text{high}}$$

---

8. 참고문헌

Original GAN

1. **Generative Adversarial Nets** Goodfellow, I., Pouget-Abadie, J., Mirza, M., et al. (2014) NeurIPS 2014 https://arxiv.org/abs/1406.2661

DCGAN

2. **Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks** Radford, A., Metz, L., & Chintala, S. (2015) ICLR 2016 https://arxiv.org/abs/1511.06434

ProGAN

3. **Progressive Growing of GANs for Improved Quality, Stability, and Variation** Karras, T., Aila, T., Laine, S., & Lehtinen, J. (2017) ICLR 2018 https://arxiv.org/abs/1710.10196

StyleGAN Series

4. **A Style-Based Generator Architecture for Generative Adversarial Networks** Karras, T., Laine, S., & Aila, T. (2018) CVPR 2019 https://arxiv.org/abs/1812.04948
5. **Analyzing and Improving the Image Quality of StyleGAN** Karras, T., Laine, S., Aittala, M., et al. (2019) CVPR 2020 https://arxiv.org/abs/1912.04958
6. **Alias-Free Generative Adversarial Networks (StyleGAN 3)** Karras, T., Aittala, M., Laine, S., et al. (2021) NeurIPS 2021 https://arxiv.org/abs/2106.12423

평가 지표

7. **Improved Techniques for Training GANs (Inception Score)** Salimans, T., Goodfellow, I., Zaremba, W., et al. (2016) NeurIPS 2016 https://arxiv.org/abs/1606.03498

8. **GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium (FID)** Heusel, M., Ramsauer, H., Unterthiner, T., et al. (2017) NeurIPS 2017 https://arxiv.org/abs/1706.08500

이론 및 분석

9. **NIPS 2016 Tutorial: Generative Adversarial Networks** Goodfellow, I. (2016) https://arxiv.org/abs/1701.00160
10. **Wasserstein GAN** Arjovsky, M., Chintala, S., & Bottou, L. (2017) ICML 2017 https://arxiv.org/abs/1701.07875
11. **Spectral Normalization for Generative Adversarial Networks** Miyato, T., Kataoka, T., Koyama, M., & Yoshida, Y. (2018) ICLR 2018 https://arxiv.org/abs/1802.05957

응용 및 변형

12. **Conditional Generative Adversarial Nets** Mirza, M., & Osindero, S. (2014) https://arxiv.org/abs/1411.1784
13. **Image-to-Image Translation with Conditional Adversarial Networks (pix2pix)** Isola, P., Zhu, J.-Y., Zhou, T., & Efros, A. A. (2016) CVPR 2017 https://arxiv.org/abs/1611.07004

다음 강의 예고: Lecture 4에서는 **CycleGAN, StyleGAN, VQ-GAN**을 다룹니다. 이미지 변환, 스타일 제어, 그리고 GAN과 이산 표현의 결합을 학습합니다.